

HW4_Problem2

January 28, 2026

1 Problem 2

In this task, you will explore strategies to help neural network architectures mitigate issues such as overfitting. To do this, you will leverage the dataset splits created in Problem 1. Your goal is to analyze the impact of each strategy when applied to a deep network architecture.

Apply the following models to the data that retain only the top half (upper portion) of each image of the Fashion MNIST database as generated up to the fourth step of Problem 1.

Train each model using an Adam optimizer, for 15 epochs with a batch size of 32, sparse categorical cross entropy as the loss function, and add accuracy as a metric. Ensure the output layer is appropriately configured for a multiclass problem.

For each of the four questions in this problem, print the model summary of each created model and generate a classification report using the scikit-learn library. Moreover, plot the training and the validation loss along with the training and the validation accuracy. In the same figure pinpoint the best epoch. Finally, comment on the obtained results. Did you notice any significant differences in performance?

1.1 2.1

1.1.1 Create and train a base model using Tensorflow/Keras library with three hidden layers of 128, 64, and 32 nodes, respectively, and add ReLU as their activation function

```
[ ]: from tensorflow import keras
```

Define the base neural network model. The model is designed to work with cropped images (top half) of shape 14x28. It flattens the 2D input into a 1D vector and passes it through three hidden dense layers with 128, 64, and 32 neurons respectively, all using ReLU activation. The output layer uses softmax activation to produce probabilities for the 10 classes.

```
[ ]: base_model = keras.Sequential([
    keras.layers.Input(shape=(14, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(64, activation="relu"),
```

```

keras.layers.Dense(32, activation="relu"),
keras.layers.Dense(10, activation="softmax")
])

```

Compile the base neural network model using the Adam optimizer, sparse categorical crossentropy as the loss function, and accuracy as the evaluation metric. This configuration is appropriate for multiclass classification tasks where the labels are provided as integers.

```

[ ]: base_model.compile(
    optimizer=keras.optimizers.Adam(),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

```

```

[ ]: base_model.summary()

```

Model: "sequential_6"

Layer (type)	Output Shape	
Param #		
flatten_6 (Flatten)	(None, 392)	
↪ 0		
dense_22 (Dense)	(None, 128)	
↪ 50,304		
dense_23 (Dense)	(None, 64)	
↪ 8,256		
dense_24 (Dense)	(None, 32)	
↪ 2,080		
dense_25 (Dense)	(None, 10)	
↪ 330		

Total params: 60,970 (238.16 KB)

Trainable params: 60,970 (238.16 KB)

Non-trainable params: 0 (0.00 B)

Train the base neural network model using the top half images (X_train_top) and the corre-

sponding training labels (y_train). The model is trained for 15 epochs with a batch size of 32. Validation is performed on the validation set (X_valid_top, y_valid) after each epoch, and the training history is stored in the variable `history_base`.

```
[ ]: history_base=base_model.fit(X_train_top, y_train, epochs=15, batch_size=32, validation_data=(X_valid_top, y_valid))
```

```
Epoch 1/15
1563/1563      2s 1ms/step -
accuracy: 0.7094 - loss: 0.8241 - val_accuracy: 0.8059 - val_loss: 0.5126
Epoch 2/15
1563/1563      2s 1ms/step -
accuracy: 0.8126 - loss: 0.4987 - val_accuracy: 0.8247 - val_loss: 0.4672
Epoch 3/15
1563/1563      2s 1ms/step -
accuracy: 0.8270 - loss: 0.4603 - val_accuracy: 0.8270 - val_loss: 0.4549
Epoch 4/15
1563/1563      2s 1ms/step -
accuracy: 0.8367 - loss: 0.4289 - val_accuracy: 0.8284 - val_loss: 0.4638
Epoch 5/15
1563/1563      2s 1ms/step -
accuracy: 0.8395 - loss: 0.4176 - val_accuracy: 0.8311 - val_loss: 0.4548
Epoch 6/15
1563/1563      2s 1ms/step -
accuracy: 0.8455 - loss: 0.4010 - val_accuracy: 0.8286 - val_loss: 0.4523
Epoch 7/15
1563/1563      2s 1ms/step -
accuracy: 0.8517 - loss: 0.3843 - val_accuracy: 0.8411 - val_loss: 0.4248
Epoch 8/15
1563/1563      2s 1ms/step -
accuracy: 0.8574 - loss: 0.3679 - val_accuracy: 0.8365 - val_loss: 0.4323
Epoch 9/15
1563/1563      2s 1ms/step -
accuracy: 0.8572 - loss: 0.3684 - val_accuracy: 0.8461 - val_loss: 0.4167
Epoch 10/15
1563/1563      2s 1ms/step -
accuracy: 0.8660 - loss: 0.3482 - val_accuracy: 0.8414 - val_loss: 0.4322
Epoch 11/15
1563/1563      2s 1ms/step -
accuracy: 0.8654 - loss: 0.3483 - val_accuracy: 0.8358 - val_loss: 0.4550
Epoch 12/15
1563/1563      2s 1ms/step -
accuracy: 0.8698 - loss: 0.3383 - val_accuracy: 0.8414 - val_loss: 0.4429
Epoch 13/15
1563/1563      2s 1ms/step -
accuracy: 0.8713 - loss: 0.3273 - val_accuracy: 0.8492 - val_loss: 0.4202
Epoch 14/15
1563/1563      2s 1ms/step -
```

```

accuracy: 0.8753 - loss: 0.3193 - val_accuracy: 0.8414 - val_loss: 0.4542
Epoch 15/15
1563/1563          2s 1ms/step -
accuracy: 0.8820 - loss: 0.3070 - val_accuracy: 0.8521 - val_loss: 0.4222

Evaluate the model on the test set

```

```

[ ]: y_pred_base = base_model.predict(X_test_top).argmax(axis=1)
print("Classification Report for Base Model:")
print(classification_report(y_test, y_pred_base, target_names=labels))

```

```

313/313          0s 663us/step
Classification Report for Base Model:

```

	precision	recall	f1-score	support
T-shirt/top	0.82	0.78	0.80	1000
Trouser	0.99	0.94	0.96	1000
Pullover	0.74	0.74	0.74	1000
Dress	0.82	0.90	0.86	1000
Coat	0.70	0.74	0.72	1000
Sandal	0.96	0.94	0.95	1000
Shirt	0.63	0.59	0.61	1000
Sneaker	0.91	0.93	0.92	1000
Bag	0.96	0.96	0.96	1000
Ankle boot	0.92	0.93	0.92	1000
accuracy			0.85	10000
macro avg	0.85	0.85	0.85	10000
weighted avg	0.85	0.85	0.85	10000

```

[ ]: plt.figure(figsize=(10, 5))
# Loop over each key in the training history dictionary and assign a style for
# plotting. The zip pairs each key with a corresponding line style.
for key, style in zip(history_base.history, ["r--", "r--.", "b-", "b-*"]):
    # The validation error is computed at the end of each epoch, while the
    # training error is computed using a running mean during each epoch,
    # so the training curve should be shifted by half an epoch to the left.
    # Adjust epochs: if key is for validation (starts with 'val_'), no shift.
    # For training keys, shift by -0.5 for better visibility
    epochs = np.array(history_base.epoch) + (0 if key.startswith("val_") else
    -0.5)
    # Plot the history values for this key with the specified style and label
    # it with the key
    plt.plot(epochs, history_base.history[key], style, label=key)

# Determine the epoch index where the validation loss is minimum
best_epoch = np.argmin(history_base.history['val_loss'])

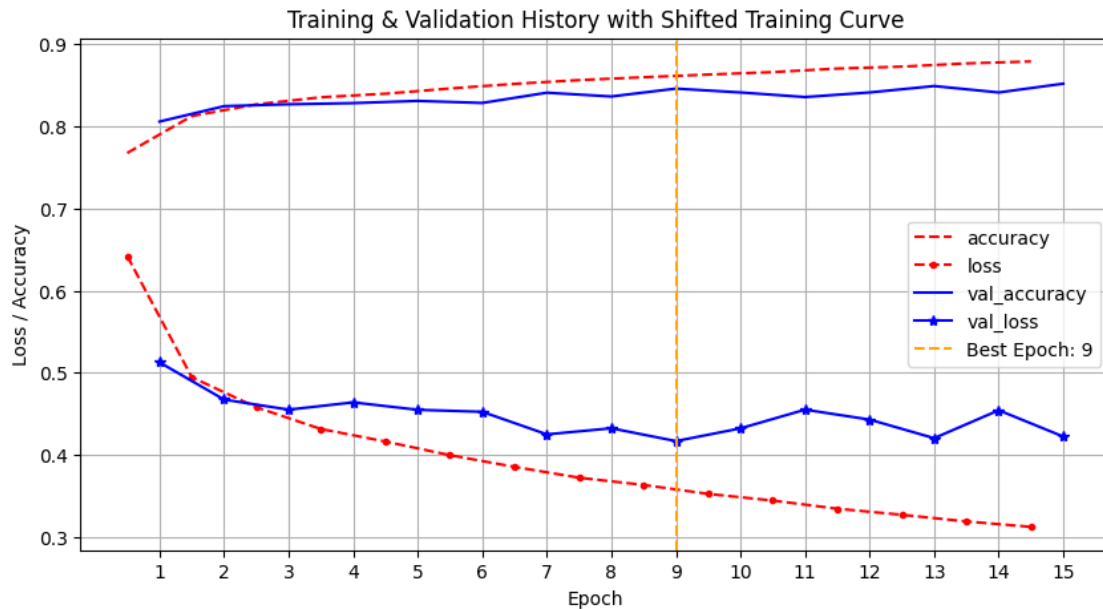
```

```

# Draw a vertical line at the best epoch (add 1 to counter for Python's
↳zero-indexing) with an orange dashed line
plt.axvline(best_epoch, color='orange', linestyle='--', label=f'Best Epoch:
↳{best_epoch + 1}')

# Set the x-ticks to represent each epoch (starting at 1) using the number of
↳epochs in the training history
plt.xticks(ticks=np.arange(len(history_base.epoch)), labels=np.arange(1,
↳len(history_base.epoch) + 1))
plt.xlabel("Epoch")
plt.ylabel("Loss / Accuracy")
plt.legend(loc="right")
plt.grid()
plt.title("Training & Validation History with Shifted Training Curve")
plt.show()

```



Best Epoch: 9

The training accuracy is steadily increasing reaching around 88%. The model learns the training data well.

The validation accuracy remains around 85% suggesting mild overfitting since the gap between training and validation performance is notable

The training loss decreases steadily, indicating that the model is effectively minimizing the error on training data

The validation loss remains steadily high, showing no major improvement. The fact that it doesn't continue decreasing in sync with training loss, means that the model learns training-specific patterns that do not help reduce validation loss, which supports the observation of overfitting.

Summary The base model sets a good baseline, achieving decent performance but also

demonstrating a classic overfitting pattern.

1.2 2.2

1.2.1 Rebuild and train the base model after adding an early stopping mechanism, with a patience argument of 3, and track the validation loss.

Set up an early stopping mechanism for training the model. The EarlyStopping callback monitors the validation loss and stops training if it does not improve for 3 consecutive epochs (`patience=3`). Additionally, it restores the model weights from the epoch with the best validation loss by setting `restore_best_weights=True`.

```
[ ]: early_stopping = keras.callbacks.EarlyStopping(monitor='val_loss', patience=3,  
↳ restore_best_weights=True)
```

Define a Sequential model using early stopping.

```
[ ]: model_early_stopping = keras.Sequential([  
    keras.layers.Input(shape=(14,28)),  
    keras.layers.Flatten(),  
    keras.layers.Dense(128, activation="relu"),  
    keras.layers.Dense(64, activation="relu"),  
    keras.layers.Dense(32, activation="relu"),  
    keras.layers.Dense(10, activation="softmax")  
])
```

Compile the model

```
[ ]: model_early_stopping.compile(  
    optimizer=keras.optimizers.Adam(),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

```
[ ]: model_early_stopping.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	
↳ Param #		
flatten_7 (Flatten)	(None, 392)	↳
↳ 0		
dense_26 (Dense)	(None, 128)	↳
↳ 50,304		

```

dense_27 (Dense)                                     (None, 64)
↳8,256
dense_28 (Dense)                                     (None, 32)
↳2,080
dense_29 (Dense)                                     (None, 10)
↳330

```

Total params: 60,970 (238.16 KB)

Trainable params: 60,970 (238.16 KB)

Non-trainable params: 0 (0.00 B)

Train the model with early stoping on top-half images

```

[ ]: history_early_stopping=model_early_stopping.fit(X_train_top, y_train,
↳epochs=15, batch_size=32,
validation_data=(X_valid_top,
↳y_valid), callbacks=[early_stopping])

```

```

Epoch 1/15
1563/1563          3s 1ms/step -
accuracy: 0.7066 - loss: 0.8286 - val_accuracy: 0.8025 - val_loss: 0.5231
Epoch 2/15
1563/1563          2s 1ms/step -
accuracy: 0.8109 - loss: 0.5021 - val_accuracy: 0.8231 - val_loss: 0.4782
Epoch 3/15
1563/1563          2s 1ms/step -
accuracy: 0.8270 - loss: 0.4541 - val_accuracy: 0.8193 - val_loss: 0.4798
Epoch 4/15
1563/1563          2s 1ms/step -
accuracy: 0.8373 - loss: 0.4328 - val_accuracy: 0.8348 - val_loss: 0.4395
Epoch 5/15
1563/1563          2s 1ms/step -
accuracy: 0.8447 - loss: 0.4106 - val_accuracy: 0.8377 - val_loss: 0.4287
Epoch 6/15
1563/1563          2s 1ms/step -
accuracy: 0.8493 - loss: 0.3973 - val_accuracy: 0.8366 - val_loss: 0.4389
Epoch 7/15
1563/1563          2s 1ms/step -
accuracy: 0.8519 - loss: 0.3844 - val_accuracy: 0.8347 - val_loss: 0.4520
Epoch 8/15
1563/1563          2s 1ms/step -

```

accuracy: 0.8568 - loss: 0.3702 - val_accuracy: 0.8363 - val_loss: 0.4385

Evaluate the model on the test set

```
[ ]: y_pred_early_stopping = model_early_stopping.predict(X_test_top).argmax(axis=1)
print("Classification Report for Early Stopping Model")
print(classification_report(y_test, y_pred_early_stopping, target_names=labels))
```

```
313/313          0s 647us/step
Classification Report for Early Stopping Model
```

	precision	recall	f1-score	support
T-shirt/top	0.83	0.73	0.78	1000
Trouser	0.98	0.93	0.95	1000
Pullover	0.71	0.69	0.70	1000
Dress	0.83	0.86	0.85	1000
Coat	0.67	0.72	0.70	1000
Sandal	0.96	0.93	0.94	1000
Shirt	0.56	0.60	0.58	1000
Sneaker	0.87	0.95	0.90	1000
Bag	0.96	0.96	0.96	1000
Ankle boot	0.94	0.88	0.91	1000
accuracy			0.83	10000
macro avg	0.83	0.83	0.83	10000
weighted avg	0.83	0.83	0.83	10000

Plot training history

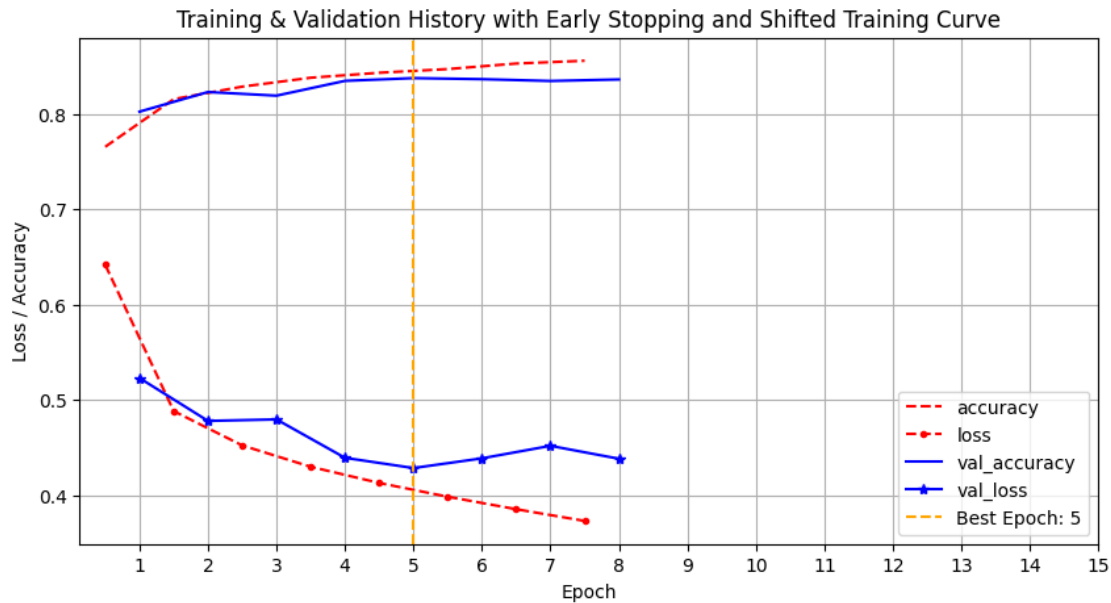
```
[ ]: plt.figure(figsize=(10,5))
for key, style in zip(history_early_stopping.history, ["r--", "r--.", "b-",
↳ "b-*"]):
    epochs = np.array(history_early_stopping.epoch) + (0 if key.
↳ startswith("val_") else -0.5)
    plt.plot(epochs, history_early_stopping.history[key], style, label=key)

best_epoch_early = np.argmin(history_early_stopping.history['val_loss'])
plt.axvline(best_epoch_early, color='orange', linestyle='--', label=f'Best_
↳ Epoch: {best_epoch_early + 1}')

plt.xticks(ticks=np.arange(len(history_base.epoch)), labels=np.arange(1,
↳ len(history_base.epoch) + 1))
plt.xlabel("Epoch")
plt.ylabel("Loss / Accuracy")
plt.legend(loc="lower right")
plt.grid()
plt.title("Training & Validation History with Early Stopping and Shifted_
↳ Training Curve")
```



```
plt.show()
```



Best Epoch: 5

The early stopping was triggered at epoch 5, preventing unnecessary training, even though it may stop before the highest training accuracy is reached.

The training accuracy steadily increases, approaching 85%. This means it does not achieve the same higher training accuracy as the base model, but avoids overfitting.

The validation accuracy maintained around 83%, which is still good, even though it is lower than the base model. The smaller gap between training and validation accuracy suggests a better generalization.

The training loss decreases quickly. Once it starts flattening out, the early stopping kicks in. The quick drop suggests that the model learns the main patterns quickly.

The validation loss drops fast until around 43%, then followed by three epochs with no improvement, setting off the early stopping.

Summary Early stopping is beneficial for preventing unnecessary training and overfitting. While the final validation accuracy is close to the base model, this approach saves training time and typically produces a model that generalizes well without over-tailoring to the training set.

1.3 2.3

1.3.1 Rebuild and train the base model after incorporating a batch normalization layer after each dense layer (do not use early stopping, and ensure the use bias argument of dense layers is set to False).

Define a Sequential model that incorporates batch normalization after each dense layer.

```
[ ]: model_batch_normalization = keras.Sequential([
    keras.layers.Input(shape=(14, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu", use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(64, activation="relu", use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(32, activation="relu", use_bias=False),
    keras.layers.BatchNormalization(),
    keras.layers.Dense(10, activation="softmax")
])
```

Compile the model

```
[ ]: model_batch_normalization.compile(
    optimizer=keras.optimizers.Adam(),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
[ ]: model_batch_normalization.summary()
```

Model: "sequential_8"

Layer (type)	Output Shape	
Param #		
flatten_8 (Flatten)	(None, 392)	
↪ 0		
dense_30 (Dense)	(None, 128)	
↪ 50,176		
batch_normalization_3	(None, 128)	
↪ 512		
(BatchNormalization)		
↪		
dense_31 (Dense)	(None, 64)	
↪ 8,192		
batch_normalization_4	(None, 64)	
↪ 256		
(BatchNormalization)		
↪		

```

dense_32 (Dense)                                     (None, 32)
↳2,048
batch_normalization_5                               (None, 32)
↳128
(BatchNormalization)
↳
dense_33 (Dense)                                     (None, 10)
↳330

```

Total params: 61,642 (240.79 KB)

Trainable params: 61,194 (239.04 KB)

Non-trainable params: 448 (1.75 KB)

Train the model with batch normalization on top-half images

```
[ ]: history_batch_normalization = model_batch_normalization.fit(
    X_train_top, y_train, epochs=15, batch_size=32,
    validation_data=(X_valid_top, y_valid)
)
```

```

Epoch 1/15
1563/1563      3s 1ms/step -
accuracy: 0.7235 - loss: 0.8007 - val_accuracy: 0.7896 - val_loss: 0.5570
Epoch 2/15
1563/1563      2s 1ms/step -
accuracy: 0.8031 - loss: 0.5287 - val_accuracy: 0.8102 - val_loss: 0.5142
Epoch 3/15
1563/1563      2s 1ms/step -
accuracy: 0.8143 - loss: 0.4944 - val_accuracy: 0.8137 - val_loss: 0.4895
Epoch 4/15
1563/1563      2s 1ms/step -
accuracy: 0.8225 - loss: 0.4763 - val_accuracy: 0.8264 - val_loss: 0.4556
Epoch 5/15
1563/1563      2s 1ms/step -
accuracy: 0.8314 - loss: 0.4490 - val_accuracy: 0.8324 - val_loss: 0.4500
Epoch 6/15
1563/1563      2s 1ms/step -
accuracy: 0.8345 - loss: 0.4373 - val_accuracy: 0.8315 - val_loss: 0.4541
Epoch 7/15
1563/1563      2s 1ms/step -
accuracy: 0.8358 - loss: 0.4312 - val_accuracy: 0.8281 - val_loss: 0.4591

```

```

Epoch 8/15
1563/1563          2s 1ms/step -
accuracy: 0.8450 - loss: 0.4162 - val_accuracy: 0.8315 - val_loss: 0.4440
Epoch 9/15
1563/1563          2s 1ms/step -
accuracy: 0.8486 - loss: 0.4036 - val_accuracy: 0.8334 - val_loss: 0.4368
Epoch 10/15
1563/1563          2s 1ms/step -
accuracy: 0.8486 - loss: 0.4038 - val_accuracy: 0.8398 - val_loss: 0.4357
Epoch 11/15
1563/1563          2s 1ms/step -
accuracy: 0.8524 - loss: 0.3860 - val_accuracy: 0.8403 - val_loss: 0.4286
Epoch 12/15
1563/1563          2s 1ms/step -
accuracy: 0.8547 - loss: 0.3840 - val_accuracy: 0.8342 - val_loss: 0.4460
Epoch 13/15
1563/1563          2s 1ms/step -
accuracy: 0.8554 - loss: 0.3761 - val_accuracy: 0.8363 - val_loss: 0.4548
Epoch 14/15
1563/1563          2s 1ms/step -
accuracy: 0.8592 - loss: 0.3705 - val_accuracy: 0.8427 - val_loss: 0.4298
Epoch 15/15
1563/1563          2s 1ms/step -
accuracy: 0.8595 - loss: 0.3656 - val_accuracy: 0.8377 - val_loss: 0.4413

```

Evaluate the model on the test set

```

[ ]: y_pred_batch_normalization = model_batch_normalization.predict(X_test_top).
      ↪argmax(axis=1)
print("Classification Report for Batch Normalization Model")
print(classification_report(y_test, y_pred_batch_normalization,
      ↪target_names=labels))

```

```

313/313          0s 747us/step
Classification Report for Batch Normalization Model
      precision    recall  f1-score   support

T-shirt/top          0.80      0.77      0.79      1000
  Trouser            0.96      0.95      0.96      1000
  Pullover           0.62      0.87      0.72      1000
    Dress            0.82      0.88      0.85      1000
     Coat            0.74      0.69      0.71      1000
    Sandal            0.96      0.94      0.95      1000
     Shirt            0.69      0.44      0.54      1000
   Sneaker            0.91      0.91      0.91      1000
      Bag            0.97      0.96      0.96      1000
  Ankle boot          0.91      0.93      0.92      1000

```

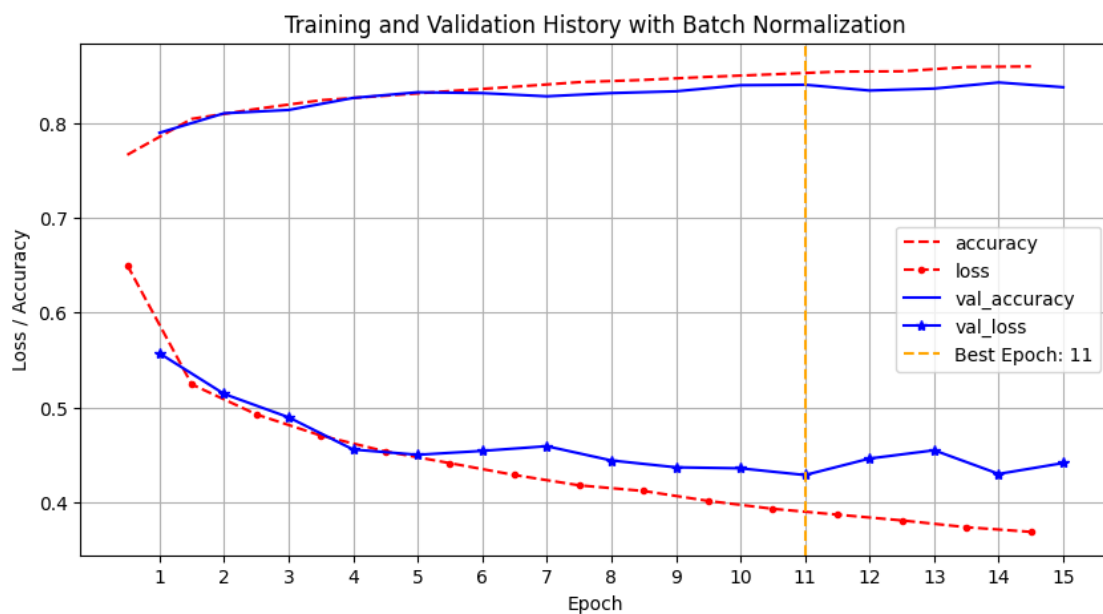
accuracy			0.83	10000
macro avg	0.84	0.83	0.83	10000
weighted avg	0.84	0.83	0.83	10000

Plot training history

```
[ ]: plt.figure(figsize=(10,5))
for key, style in zip(history_batch_normalization.history, ["r--", "r--.", "b-", "b-*"]):
    epochs = np.array(history_batch_normalization.epoch) + (0 if key.
    ↳startswith("val_") else -0.5)
    plt.plot(epochs, history_batch_normalization.history[key], style, label=key)

best_epoch_batch_normalization = np.argmin(history_batch_normalization.
    ↳history['val_loss'])
plt.axvline(best_epoch_batch_normalization, color='orange', linestyle='--',
    ↳label=f'Best Epoch: {best_epoch_batch_normalization + 1}')

plt.xticks(ticks=np.arange(len(history_base.epoch)), labels=np.arange(1,
    ↳len(history_base.epoch) + 1))
plt.xlabel("Epoch")
plt.ylabel("Loss / Accuracy")
plt.legend(loc="right")
plt.grid()
plt.title("Training and Validation History with Batch Normalization")
plt.show()
```



Best Epoch: 11

The training accuracy steadily increases, approaching 88%. This is similar to the base model.

The validation accuracy maintained around 85% which is similar with the best validation accuracy seen in the base model. It often remains more stable and smooth due to batch normalization.

The training loss decreases smoothly, and consistently, reflecting the stabilizing effect of batch normalization. When activations are normalized, gradients flow more predictably, and the network often avoids erratic jumps in loss.

The validation loss Gradually declines to about 45%, mirroring the consistent improvement in validation accuracy. This indicates a more stable learning curve compared to the base model, which sometimes shows a plateau or fluctuations earlier. ##### Summary Batch normalization often leads to a smoother and sometimes quicker convergence because it normalizes activations layer by layer. Here, it yields performance roughly comparable to the base model in terms of final accuracy, but it's typically more robust, and less sensitive to initial conditions.

1.4 2.4

1.4.1 Rebuild and train the base model after adding a dropout layer with a rate of 0.50 after each dense layer (do not use early stopping or batch normalization)

Define a Sequential model that incorporates dropout layers to help reduce overfitting.

```
[ ]: model_dropout = keras.Sequential([
    keras.layers.Input(shape=(14, 28)),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(64, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(10, activation="softmax")
])
```

Compile the model

```
[ ]: model_dropout.compile(
    optimizer=keras.optimizers.Adam(),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

Train the model on top-half images

```
[ ]: history_dropout = model_dropout.fit(
    X_train_top, y_train, epochs=15, batch_size=32,
    validation_data=(X_valid_top, y_valid)
)
```

Epoch 1/15

1563/1563

3s 1ms/step -

```

accuracy: 0.3950 - loss: 1.6067 - val_accuracy: 0.7516 - val_loss: 0.7003
Epoch 2/15
1563/1563          2s 1ms/step -
accuracy: 0.6661 - loss: 0.8987 - val_accuracy: 0.7667 - val_loss: 0.6312
Epoch 3/15
1563/1563          2s 1ms/step -
accuracy: 0.7093 - loss: 0.8088 - val_accuracy: 0.7755 - val_loss: 0.6021
Epoch 4/15
1563/1563          2s 1ms/step -
accuracy: 0.7223 - loss: 0.7745 - val_accuracy: 0.7791 - val_loss: 0.5784
Epoch 5/15
1563/1563          2s 1ms/step -
accuracy: 0.7324 - loss: 0.7482 - val_accuracy: 0.7805 - val_loss: 0.5750
Epoch 6/15
1563/1563          2s 1ms/step -
accuracy: 0.7381 - loss: 0.7261 - val_accuracy: 0.7899 - val_loss: 0.5699
Epoch 7/15
1563/1563          2s 1ms/step -
accuracy: 0.7423 - loss: 0.7269 - val_accuracy: 0.7904 - val_loss: 0.5557
Epoch 8/15
1563/1563          2s 1ms/step -
accuracy: 0.7501 - loss: 0.7000 - val_accuracy: 0.7916 - val_loss: 0.5483
Epoch 9/15
1563/1563          2s 1ms/step -
accuracy: 0.7488 - loss: 0.6973 - val_accuracy: 0.8045 - val_loss: 0.5399
Epoch 10/15
1563/1563          2s 1ms/step -
accuracy: 0.7569 - loss: 0.6810 - val_accuracy: 0.8055 - val_loss: 0.5436
Epoch 11/15
1563/1563          2s 1ms/step -
accuracy: 0.7599 - loss: 0.6711 - val_accuracy: 0.7986 - val_loss: 0.5481
Epoch 12/15
1563/1563          2s 1ms/step -
accuracy: 0.7609 - loss: 0.6713 - val_accuracy: 0.8023 - val_loss: 0.5305
Epoch 13/15
1563/1563          2s 1ms/step -
accuracy: 0.7641 - loss: 0.6635 - val_accuracy: 0.8021 - val_loss: 0.5344
Epoch 14/15
1563/1563          2s 1ms/step -
accuracy: 0.7681 - loss: 0.6650 - val_accuracy: 0.8102 - val_loss: 0.5205
Epoch 15/15
1563/1563          2s 1ms/step -
accuracy: 0.7667 - loss: 0.6626 - val_accuracy: 0.8097 - val_loss: 0.5268

```

Evaluate the model on the test set

```
[ ]: y_pred_dropout = model_dropout.predict(X_test_top).argmax(axis=1)
      print("Classification Report for Dropout Regularization Model")
```

```
print(classification_report(y_test, y_pred_dropout, target_names=labels))
```

```
313/313          0s 651us/step
Classification Report for Dropout Regularization Model
```

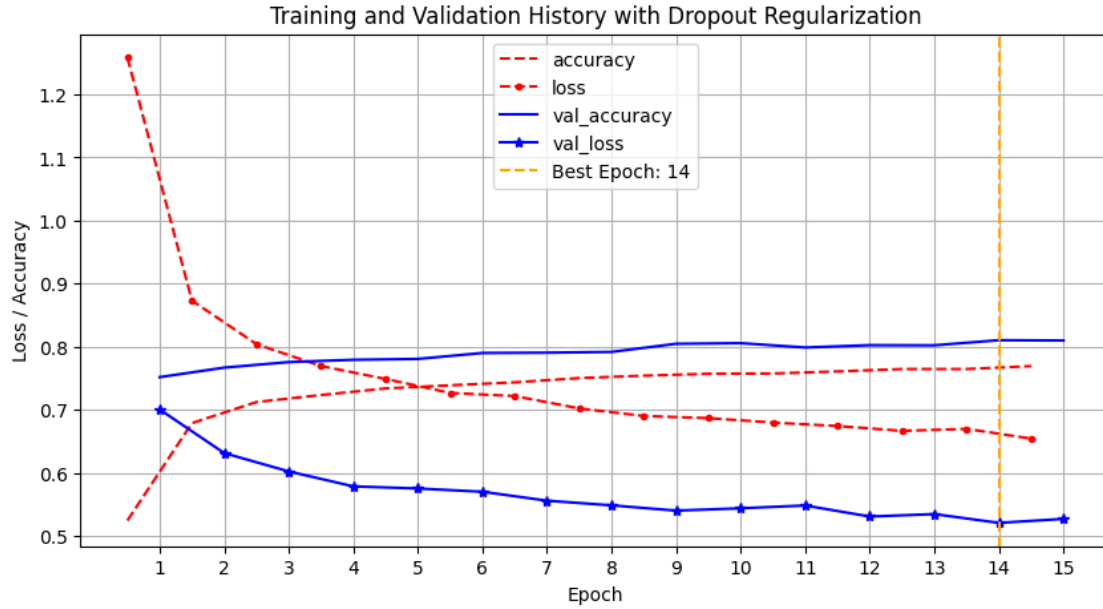
	precision	recall	f1-score	support
T-shirt/top	0.78	0.77	0.77	1000
Trouser	0.96	0.92	0.94	1000
Pullover	0.63	0.66	0.64	1000
Dress	0.84	0.79	0.81	1000
Coat	0.55	0.85	0.67	1000
Sandal	0.95	0.93	0.94	1000
Shirt	0.51	0.26	0.35	1000
Sneaker	0.90	0.91	0.90	1000
Bag	0.94	0.95	0.95	1000
Ankle boot	0.91	0.93	0.92	1000
accuracy			0.80	10000
macro avg	0.80	0.80	0.79	10000
weighted avg	0.80	0.80	0.79	10000

Plot training history

```
[ ]: plt.figure(figsize=(10,5))
for key, style in zip(history_dropout.history, ["r--", "r--.", "b-", "b-*"]):
    epochs = np.array(history_dropout.epoch) + (0 if key.startswith("val_")
    ↪ else -0.5)
    plt.plot(epochs, history_dropout.history[key], style, label=key)

best_epoch_dropout = np.argmin(history_dropout.history['val_loss'])
plt.axvline(best_epoch_dropout, color='orange', linestyle='--', label=f'Best
    ↪ Epoch: {best_epoch_dropout + 1}')

plt.xticks(ticks=np.arange(len(history_base.epoch)), labels=np.arange(1,
    ↪ len(history_base.epoch) + 1))
plt.xlabel("Epoch")
plt.ylabel("Loss / Accuracy")
plt.legend(loc="upper center")
plt.grid()
plt.title("Training and Validation History with Dropout Regularization")
plt.show()
```

Best Epoch: 14

The training accuracy stays behind the other models, ending up around 78%. This lower value is due to the aggressive 50% dropout rate, which prevents the model from fully learning the data by randomly deactivating half of the neurons with each forward pass.

The validation accuracy shows a gradual improvement, eventually reaching around 80%. This suggests that although the model is constrained by dropout during training, it manages to generalize reasonably well to unseen data.

The training loss drops quickly in the early epochs and continues to decline, stabilizing at approximately 0.65 by the final epoch. This indicates that despite the regularization, the model is effectively minimizing the error on the training data.

The validation loss decreases in a smooth, steady manner over the epochs, which is indicative of a consistent learning process and stable generalization performance throughout training. #####

Summary The dropout model, with a 50% dropout rate after each dense layer, significantly restricts the network's capacity, as evidenced by the training accuracy staying below 78%. Nevertheless, the model is able to achieve a validation accuracy of about 80%, demonstrating that the dropout is effective at tackling overfitting. The rapid initial drop in training loss, which then levels off at around 0.65, along with the smoothly declining validation loss, highlights the trade-off: while dropout helps maintain a stable learning curve and good generalization, it also limits the ultimate learning capacity of the model.